
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
D'AL-HOCEIMA (ENSAH)

Documentation Technique

Gestion des Soutenances PFE



Version	1.0.0-SNAPSHOT
Date	Mai 2026
Statut	Production

Réalisée par : El-Hajjioui Chaymae

Encadré par : Pr.Cherradi Mohamed

Table des matières

1. Vue d'ensemble du projet	5
1.1 Contexte.....	5
1.2 Problème résolu.....	5
1.3 Fonctionnalités principales	5
1.4 Flux de travail global	5
2. Architecture du système	6
2.1 Pattern architectural.....	6
2.2 Vue des composants	6
2.3 Conventions de nommage	6
3. Stack technique.....	7
3.1 Dépendances principales	7
3.2 Prérequis de build	7
3.3 Modules iText 8	7
4. Prérequis système.....	8
4.1 Environnement de développement.....	8
4.2 Environnement de production (minimal)	8
4.3 Système de fichiers	8
5. Structure du projet	9
5.1 Arborescence principale	9
5.2 Ressources	9
6. Modèle de données	10
6.1 Entités et relations	10
6.2 Détail des entités	10
Filiere : Filière académique.....	10
Etudiant.....	10
Professeur	10
Soutenance	11
MembreJury — Table de jointure.....	11
6.3 Énumérations.....	11
7. Couche contrôleurs — API & Routes	12
7.1 MainController — Routes de navigation	12
7.2 ExportController — Génération de fichiers.....	12
7.3 GlobalControllerAdvice.....	13

8. Couche service — Logique métier	14
8.1 ExcellImportService	14
Types de feuilles reconnues	14
Comportements clés.....	14
8.2 MatchingService	14
Algorithme de matching	14
8.3 PlanningService.....	14
Algorithme de génération.....	15
Paramètres configurables	15
8.4 DashboardService — Métriques.....	15
8.5 ExcelExportService et PdfExportService	15
9. Moteur de vérification	16
9.1 Interface VerificationRule	16
9.2 Tableau des 8 règles	16
9.3 Structure Anomalie (Java Record)	16
9.4 Extensibilité — Ajouter une règle.....	16
10. Couche de persistance — Repositories.....	17
10.1 EtudiantRepository	17
10.2 ProfesseurRepository	17
10.3 SoutenanceRepository.....	17
10.4 SalleRepository	17
11. Interface utilisateur — Templates Thymeleaf	18
11.1 Vue d'ensemble	18
11.2 Graphiques Chart.js (dashboard.html)	18
11.3 Ressources statiques	18
12. Référence de configuration	19
12.1 application.properties (développement)	19
12.3 Variables d'environnement	19
13. Build & Déploiement.....	20
13.1 Build Maven	20
13.2 Exécution locale (développement)	20
14. Guide de développement.....	21
14.1 Setup de l'environnement	21
14.2 Annotations Lombok utilisées	21

14.3 Ajouter une nouvelle entité.....	21
14.4 Ajouter une règle de vérification.....	21
14.5 Tests.....	21
14.6 Accès à la base de données (développement)	22
15. Sécurité.....	23
15.1 État actuel	23
15.2 Mesures minimales recommandées (production)	23
1. Désactiver la console H2	23
2. Ajouter Spring Security	23
3. Restreindre l'accès réseau	23
4. Sauvegardes quotidiennes de la base H2	23
15.3 Validation des entrées	23
16. Limitations connues & Évolutions futures	24
16.1 Limitations techniques actuelles	24
16.2 Évolutions fonctionnelles suggérées	24
16.3 Migration PostgreSQL recommandée	24
Annexe — Glossaire.....	25

1. Vue d'ensemble du projet

1.1 Contexte

ENSAH Gestion des Soutenances est une application web interne développée pour l'École Nationale des Sciences Appliquées d'Al-Hoceima. Elle automatise l'organisation complète des soutenances de Projets de Fin d'Études (PFE) : importation des données, affectation des encadrants, génération des plannings et vérification de la cohérence des données.

1.2 Problème résolu

Sans cet outil, la coordination des soutenances implique des opérations manuelles coûteuses en temps et sources d'erreurs :

- Saisie manuelle de données depuis plusieurs fichiers Excel
- Affectation manuelle des encadrants selon leurs spécialités
- Construction manuelle d'un planning en évitant les conflits de salles et de professeurs
- Vérification laborieuse de la complétude des jurys

L'application automatise l'ensemble de ce processus.

1.3 Fonctionnalités principales

Fonctionnalité	Description
Import Excel	Chargement en masse via fichiers .xlsx (étudiants, professeurs, salles, filières, jours)
Matching encadrants	Algorithme d'affectation basé sur la correspondance de spécialités et l'équilibrage de charge
Génération de planning	Création automatique de créneaux horaires avec jury, salle et gestion des conflits
Vérification des données	Moteur à 8 règles métier détectant erreurs et avertissements
Export	Génération de documents Excel (.xlsx) et PDF pour le planning et les affectations
Tableau de bord	Statistiques globales avec graphiques dynamiques (Chart.js)

1.4 Flux de travail global

Le flux de traitement suit cinq étapes séquentielles :

- **IMPORT** : Chargement des fichiers Excel (filières, professeurs, étudiants, salles, jours)
- **MATCHING** : Affectation des encadrants par score de spécialités et équilibrage de charge
- **PLANNING** : Génération des créneaux, attribution des salles et constitution des jurys
- **VÉRIFICATION** : Exécution des 8 règles métier pour détecter anomalies et conflits
- **EXPORT** : Production des documents Excel et PDF finaux

2. Architecture du système

2.1 Pattern architectural

L'application suit une architecture MVC en couches (Model-View-Controller), typique des applications Spring Boot avec rendu serveur (Server-Side Rendering). Chaque couche dispose de responsabilités strictement définies.

Couche	Composants	Rôle
Contrôleur	MainController, ExportController, VerificationController, GlobalControllerAdvice	Traitement HTTP, orchestration des requêtes
Service	7 services + 8 règles de vérification	Logique métier, algorithmes, exports
Repository	7 interfaces JpaRepository	Accès données via Hibernate/JPA
Persistence	H2 Database (fichier ./data/soutenances.mv.db)	Stockage relationnel embarqué

2.2 Vue des composants

Module	Technologies	Artefacts
Couche présentation	Thymeleaf, HTML5, CSS3, Bootstrap	11 templates HTML
Assets statiques	CSS personnalisé, Chart.js	css/main.css, js/main.js
Couche contrôleur	Spring MVC (@Controller)	4 contrôleurs
Couche service	Spring (@Service, @Transactional)	7 services + 8 règles
Couche données	Spring Data JPA, Hibernate 6	7 entités, 7 repositories
Export Excel	Apache POI 5.2.5	Fichiers .xlsx
Export PDF	iText 8.0.3	Fichiers .pdf

2.3 Conventions de nommage

Couche	Convention	Exemple
Entités	PascalCase	Etudiant, MembreJury
Repositories	{Entité}Repository	EtudiantRepository
Services	{Domaine}Service + {Domaine}ServiceImpl	MatchingService / MatchingServiceImpl
Contrôleurs	{Domaine}Controller	ExportController
Templates	camelCase.html	planning.html, matching.html

3. Stack technique

3.1 Dépendances principales

Composant	Artefact Maven	Version	Rôle
Spring Boot	spring-boot-starter-parent	3.2.0	Framework principal
Spring MVC	spring-boot-starter-web	3.2.0	Contrôleurs web & REST
Thymeleaf	spring-boot-starter-thymeleaf	3.2.0	Moteur de templates SSR
Spring Data JPA	spring-boot-starter-data-jpa	3.2.0	ORM / accès base de données
Validation	spring-boot-starter-validation	3.2.0	Validation des beans (JSR-380)
H2 Database	com.h2database:h2	Latest	Base de données embarquée
Hibernate	(inclus dans Spring Data JPA)	6.x	ORM - génération SQL
Apache POI	org.apache.poi:poi-ooxml	5.2.5	Lecture/écriture .xlsx
iText Core	com.itextpdf:itext-core	8.0.3	Génération de fichiers PDF
Lombok	org.projectlombok:lombok	Latest	Génération de code boilerplate
Jackson	jackson-databind	Latest	Sérialisation JSON
DevTools	spring-boot-devtools	3.2.0	Rechargement à chaud (dev)

3.2 Prérequis de build

Outil	Version minimale	Remarque
Java JDK	17 (LTS)	Adoptium Temurin recommandé
Maven	3.8+	Inclus dans les IDE majeurs
Navigateur	Chrome 90+, Firefox 88+, Edge 90+	Pour l'interface utilisateur

3.3 Modules iText 8

L'application utilise le BOM itext-core qui regroupe les modules suivants :

- ``kernel`` : Primitives PDF de bas niveau
- ``layout`` : Mise en page (tableaux, paragraphes)
- ``io`` : Lectures/écritures de flux

4. Prérequis système

4.1 Environnement de développement

```
JDK 17+      → https://adoptium.net/
Maven 3.8+   → https://maven.apache.org/
IDE          → IntelliJ IDEA (recommandé), Eclipse, VS Code
```

4.2 Environnement de production (minimal)

Ressource	Minimum	Recommandé
OS	Linux / Windows Server	Ubuntu 22.04
JRE	Java 17+	OpenJDK 17 Temurin
RAM	512 MB	1 GB+
Disque	200 MB (app)	1 GB+ (app + base H2 + backups)
Réseau	Port 8080	

4.3 Système de fichiers

L'application nécessite un répertoire `./data/` accessible en écriture depuis son répertoire d'exécution pour stocker la base H2.

```
# Arborescence minimale en production
/opt/soutenance/
├─ soutenance-app-1.0.0-SNAPSHOT.jar
├─ data/
│   └─ soutenances.mv.db      ← Base de données H2
│   └─ soutenances.trace.db  ← Journal H2
```

5. Structure du projet

Le projet Maven contient 44 fichiers sources Java, 11 templates HTML et 2 ressources statiques.

5.1 Arborescence principale

```
soutenance-app/
├─ pom.xml                ← Racine Maven
├─ data/                  ← Descripteur de build
├─ src/main/java/ma/ensah/soutenance/
│   └─ SoutenanceApplication.java ← Base de données H2 (auto-crée)
│       └─ controller/
│           └─ MainController.java ← Point d'entrée Spring Boot
│           └─ ExportController.java ← Routes principales
│           └─ VerificationController.java ← Routes export Excel/PDF
│           └─ GlobalControllerAdvice.java ← Route vérification
│       └─ entity/          ← Attributs globaux
│       └─ repository/      ← 7 entités JPA
│       └─ service/
│           └─ impl/        ← 7 interfaces JpaRepository
│           └─ verification/rules/ ← 7 implémentations
│                                   ← 8 règles métier
```

5.2 Ressources

```
src/main/resources/
├─ application.properties ← Configuration principale
├─ templates/             ← 11 templates Thymeleaf
```



```

├── layout.html    dashboard.html    import.html
├── etudiants.html professeurs.html  salles.html
├── matching.html  planning.html     verification.html
├── static/
│   ├── css/main.css
│   └── js/main.js

```

6. Modèle de données

6.1 Entités et relations

Le modèle comporte 7 entités JPA interconnectées. La relation centrale est la soutenance, qui relie un étudiant, une salle et un jury de 3 professeurs.

6.2 Détail des entités

Filiere : Filière académique

Champ	Type SQL	Contraintes	Description
id	BIGINT	PK, AUTO_INCREMENT	Identifiant technique
code	VARCHAR	UNIQUE, NOT NULL	Code court (ex: GI, ID, TDIA)
nom	VARCHAR	UNIQUE, NOT NULL	Nom complet de la filière
description	VARCHAR	NULL	Description optionnelle

Etudiant

Champ	Type SQL	Contraintes	Description
id	BIGINT	PK, AUTO_INCREMENT	Identifiant technique
cne	VARCHAR	NOT NULL	Code National de l'Étudiant
nom	VARCHAR	NOT NULL	Nom de famille
prenom	VARCHAR	NOT NULL	Prénom
email	VARCHAR	NULL	Adresse e-mail
sujet_stage	TEXT	NULL	Intitulé du sujet de PFE
mots_cles_stage	TEXT	NULL	Mots-clés pour le matching (virgule-séparés)
entreprise	VARCHAR	NULL	Entreprise d'accueil
filiere_id	BIGINT	FK → Filiere	Filière de l'étudiant
encadrant_id	BIGINT	FK → Professeur, NULL	Encadrant (null avant matching)

Professeur

Champ	Type SQL	Contraintes	Description
id	BIGINT	PK, AUTO_INCREMENT	Identifiant technique

Champ	Type SQL	Contraintes	Description
nom	VARCHAR	NOT NULL	Nom de famille
prenom	VARCHAR	NOT NULL	Prénom
email	VARCHAR	NULL	Adresse e-mail
grade	VARCHAR	NULL	Grade académique
specialites	TEXT	NULL	Domaines de spécialité (virgule-séparés)
chargeMaxJury	INT	défaut 10	Nombre maximum de jurys acceptés

Soutenance

Champ	Type SQL	Contraintes	Description
id	BIGINT	PK, AUTO_INCREMENT	Identifiant technique
etudiant_id	BIGINT	FK → Etudiant	Étudiant concerné
salle_id	BIGINT	FK → Salle	Salle attribuée
date	DATE	NOT NULL	Date de la soutenance
heureDebut	TIME	NOT NULL	Heure de début
heureFin	TIME	NOT NULL	Heure de fin
statut	ENUM	NOT NULL	PLANIFIEE EN_COURS TERMINEE ANNULEE

MembreJury — Table de jointure

Champ	Type SQL	Contraintes	Description
id	BIGINT	PK, AUTO_INCREMENT	Identifiant technique
soutenance_id	BIGINT	FK → Soutenance	Soutenance concernée
professeur_id	BIGINT	FK → Professeur	Professeur membre du jury
role	ENUM	NOT NULL	ENCADRANT PRESIDENT EXAMINATEUR

6.3 Énumérations

Enum	Valeurs	Usage
StatutSoutenance	PLANIFIEE, EN_COURS, TERMINEE, ANNULEE	Cycle de vie d'une soutenance
RoleJury	ENCADRANT, PRESIDENT, EXAMINATEUR	Rôle d'un professeur dans le jury
Severite	ERROR, WARNING	Niveau d'une anomalie de vérification

7. Couche contrôleurs — API & Routes

i NOTE

L'application utilise exclusivement le protocole HTTP (GET/POST via formulaires HTML). Il n'y a pas d'API REST JSON dans la version actuelle.

7.1 MainController — Routes de navigation

Méthode	Route	Type	Description	Vue
GET	/	Page	Tableau de bord avec statistiques	dashboard
GET	/import	Page	Formulaire d'import de fichier	import
POST	/import	Action	Traite le fichier Excel uploadé	redirect /import
GET	/matching	Page	Statistiques d'affectation	matching
POST	/matching/generer	Action	Lance l'algorithme de matching	redirect /matching
POST	/matching/reinitialiser	Action	Réinitialise toutes les affectations	redirect /matching
GET	/planning	Page	Vue du planning de soutenances	planning
POST	/planning/generer	Action	Génère le planning automatiquement	redirect /planning
POST	/planning/supprimer	Action	Supprime tous les plannings	redirect /planning
GET	/etudiants	Page	Liste des étudiants	etudiants
GET	/professeurs	Page	Liste des professeurs avec charge	professeurs
GET	/salles	Page	Liste et gestion des salles	salles
GET	/verification	Page	Lance les 8 règles et affiche les résultats	verification

7.2 ExportController — Génération de fichiers

Route	Content-Type	Description
/export/planning/excel	application/vnd.openxmlformats...	Planning en Excel
/export/planning/pdf	application/pdf	Planning en PDF
/export/matching/excel	application/vnd.openxmlformats...	Affectations en Excel
/export/matching/pdf	application/pdf	Affectations en PDF
/export/template/salles	application/vnd.openxmlformats...	Template vierge — salles
/export/template/jours	application/vnd.openxmlformats...	Template vierge — jours

Toutes les routes retournent un `ResponseEntity<byte[]>` avec l'en-tête Content-Disposition: attachment; filename="...".

7.3 GlobalControllerAdvice

Fournit l'attribut `requestUri` à tous les templates Thymeleaf, utilisé pour mettre en surbrillance l'élément de menu actif dans la barre de navigation.

8. Couche service — Logique métier

8.1 ExcellImportService

Fichier : `service/impl/ExcellImportServiceImpl.java` (476 lignes)

Responsabilité : Lire un fichier `.xlsx` et persister les entités correspondantes.

Méthode principale :

```
importResult importerFichier(MultipartFile fichier)
```

Types de feuilles reconnues

Nom de feuille (insensible à la casse)	Entité créée
filieres	Filiere
professeurs	Professeur
salles	Salle
etudiants ou nom de filière	Etudiant
jours	JourSoutenance

Comportements clés

- Détection automatique des en-têtes de colonnes (insensible à la casse)
- Support de deux formats de professeurs : standard et Encadrant|Discipline
- Création automatique de la filière si le nom de feuille est inconnu
- Validation des dates (format Excel natif ET texte ISO YYYY-MM-DD)
- Idempotence : mise à jour si l'entité existe déjà (par cne, nom+prénom, ou code)

8.2 MatchingService

Fichier : `service/impl/MatchingServiceImpl.java` (145 lignes)

Responsabilité : Affecter un encadrant à chaque étudiant non encore affecté via un algorithme de scoring.

Algorithme de matching

```
POUR CHAQUE étudiant sans encadrant :
1. Extraire les mots-clés du sujet de stage
2. POUR CHAQUE professeur disponible (charge < chargeMax) :
  a. score = nombre de mots-clés du sujet présents dans les spécialités du prof
  b. Pondérer par la charge actuelle (moins chargé → bonus)
3. Affecter le professeur avec le score le plus élevé
4. Si aucun professeur disponible → enregistrer un échec
```

8.3 PlanningService

Fichier : service/impl/PlanningServiceImpl.java (180 lignes)

Responsabilité : Générer des séances de soutenance avec créneaux, salles et jurys.

Algorithme de génération

```
POUR CHAQUE JourSoutenance actif :
  POUR CHAQUE Salle disponible :
    Générer les créneaux (09:00-10:00, 10:00-11:00, ..., 17:00-18:00)
  POUR CHAQUE créneau :
    SI salle libre à ce créneau :
      Trouver un étudiant avec encadrant, sans soutenance planifiée
      Constituer le jury :
        - ENCADRANT = encadrant de l'étudiant
        - PRESIDENT = professeur sans conflit horaire (le moins chargé)
        - EXAMINATEUR = professeur sans conflit horaire (le moins chargé)
      Créer la Soutenance + les 3 MembreJury
```

Paramètres configurables

Paramètre	Valeur par défaut	Propriété
Durée d'une soutenance	60 minutes	app.soutenance.duree-minutes
Heure de début de journée	08h00	app.soutenance.heure-debut
Heure de fin de journée	18h00	app.soutenance.heure-fin
Taille du jury	3 membres	app.soutenance.jury-size

8.4 DashboardService — Métriques

Métrique	Type	Description
nbEtudiants	int	Nombre total d'étudiants
nbProfesseurs	int	Nombre total de professeurs
nbSoutenances	int	Nombre de soutenances planifiées
nbSalles	int	Nombre de salles
tauxAffectation	double	% étudiants avec encadrant
tauxPlanification	double	% étudiants avec soutenance planifiée
soutenancesParFiliere	Map	Comptage par filière
chargeParProf	Map	Nombre d'encadrés par professeur
juryParProf	Map	Nombre de participations jury par prof

8.5 ExcelExportService et PdfExportService

Service	Méthodes	Format de sortie
ExcelExportService	exporterPlanningExcel(), exporterMatchingExcel()	byte[] → .xlsx via Apache POI

Service	Méthodes	Format de sortie
PdfExportService	exporterPlanningPdf(), exporterMatchingPdf()	byte[] → .pdf via iText 8

9. Moteur de vérification

9.1 Interface VerificationRule

```
public interface VerificationRule {
    String getNom();
    String getCategorie(); // "AFFECTATION" ou "PLANNING"
    List<Anomalie> verifier();
}
```

9.2 Tableau des 8 règles

Classe	Catégorie	Sévérité	Condition détectée
EtudiantSansEncadrantRule	AFFECTATION	ERROR	Étudiant sans encadrant affecté
DistributionEncadrementRule	AFFECTATION	WARNING	Déséquilibre de charge (écart > seuil)
ConflitSalleRule	PLANNING	ERROR	Deux soutenances dans la même salle au même créneau
DoublonProfesseurRule	PLANNING	ERROR	Professeur dans deux jurys simultanés
ConflitRoleJuryRule	PLANNING	ERROR	Même rôle attribué deux fois dans un jury
CreneauHoraireRule	PLANNING	ERROR	Chevauchement de créneaux horaires
PauseInsuffisanteRule	PLANNING	WARNING	Pause < 15 min entre deux soutenances d'un même prof
JuryIncompletRule	PLANNING	ERROR	Jury avec moins de 3 membres

9.3 Structure Anomalie (Java Record)

```
record Anomalie(
    Severite severite, // ERROR ou WARNING
    String categorie, // "AFFECTATION" ou "PLANNING"
    String regle, // Nom de la classe de règle
    String message, // Description lisible
    String contexte // Entités concernées (ex: "Étudiant: Dupont Jean")
)
```

9.4 Extensibilité — Ajouter une règle

1. Créer la classe dans service/verification/rules/ en implémentant VerificationRule
2. Injecter les repositories nécessaires via le constructeur (@Autowired)

3. Implémenter `verifier()` pour retourner `List<Anomalie>`
4. Enregistrer la règle dans `VerificationServiceImpl`

10. Couche de persistance — Repositories

Tous les repositories héritent de `JpaRepository<Entité, Long>` et bénéficient automatiquement des opérations CRUD standard (`save`, `findById`, `findAll`, `delete`, `count`).

10.1 EtudiantRepository

```
Optional<Etudiant> findByCne(String cne);
List<Etudiant> findByFiliereId(Long filiereId);
List<Etudiant> findByEncadrantIsNull(); // Étudiants sans encadrant
List<Etudiant> findByEncadrantId(Long profId); // Étudiants d'un prof
```

10.2 ProfesseurRepository

```
Optional<Professeur> findByNomIgnoreCaseAndPrenomIgnoreCase(String nom, String prenom);
List<Professeur> findBySpecialitesContainingIgnoreCase(String keyword);

// Requête custom : professeurs triés par charge jury croissante
@Query("SELECT p FROM Professeur p LEFT JOIN MembreJury mj ON mj.professeur = p " +
        "GROUP BY p ORDER BY COUNT(mj) ASC")
List<Professeur> findAllOrderByChargeAsc();
```

10.3 SoutenanceRepository

```
List<Soutenance> findByDate(LocalDate date);
List<Soutenance> findByDateOrderByHeureDebutAsc(LocalDate date);
Optional<Soutenance> findByEtudiantId(Long etudiantId);
List<Soutenance> findBySalleIdAndDate(Long salleId, LocalDate date);

// Requête custom – conflit professeur
@Query("SELECT s FROM Soutenance s JOIN s.membresJury mj WHERE mj.professeur.id = :profId")
List<Soutenance> findByProfesseurAndDate(Long profId, LocalDate date);
```

10.4 SalleRepository

```
Optional<Salle> findByNomIgnoreCase(String nom);
List<Salle> findByDisponibleTrue();

// Salles libres pour un créneau donné (sans conflit)
@Query("SELECT s FROM Salle s WHERE s.disponible = true" +
        " AND s.id NOT IN (" +
        "     SELECT sout.salle.id FROM Soutenance sout" +
        "     WHERE sout.date = :date" +
        "     AND sout.heureDebut < :heureFin" +
        "     AND sout.heureFin > :heureDebut" +
        ")")
List<Salle> findSallesDisponibles(LocalDate date, LocalTime h1, LocalTime h2);
```

11. Interface utilisateur — Templates Thymeleaf

11.1 Vue d'ensemble

Template	Route	Description
layout.html	—	Template maître (navigation, structure globale)
dashboard.html	/	Tableau de bord avec statistiques et graphiques Chart.js
import.html	/import	Formulaire d'upload de fichier Excel
etudiants.html	/etudiants	Tableau de tous les étudiants avec encadrant
professeurs.html	/professeurs	Tableau des professeurs avec charge jury
salles.html	/salles	Liste et gestion des salles
matching.html	/matching	Statistiques et déclenchement du matching
planning.html	/planning	Planning des soutenances par jour
verification.html	/verification	Rapport d'anomalies avec sévérité colorée

11.2 Graphiques Chart.js (dashboard.html)

- Diagramme en anneau — répartition affectation / non-affecté
- Diagramme en barres — soutenances par filière
- Diagramme en barres — charge jury par professeur

11.3 Ressources statiques

Fichier	Rôle
static/css/main.css	Styles personnalisés (complément Bootstrap)
static/js/main.js	Initialisation des graphiques Chart.js, comportements UI

12. Référence de configuration

12.1 application.properties (développement)

```
# — Identification —————
spring.application.name=ENSAH Gestion Soutenances

# — Base de données H2 —————
spring.datasource.url=jdbc:h2:file:./data/soutenances;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update

# — Console H2 (DÉSACTIVER en production) —————
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# — Thymeleaf —————
spring.thymeleaf.cache=false
```



```
# — Upload —
spring.servlet.multipart.max-file-size=20MB
spring.servlet.multipart.max-request-size=20MB

# — Paramètres métier —
app.soutenance.duree-minutes=60
app.soutenance.heure-debut=8
app.soutenance.heure-fin=18
app.soutenance.jury-size=3

# — Logging —
logging.level.ma.ensah=DEBUG
logging.level.org.hibernate.SQL=DEBUG
```

12.3 Variables d'environnement

Variable	Exemple	Description
SERVER_PORT	8080	Port d'écoute
SPRING_PROFILES_ACTIVE	prod	Profil Spring actif
SPRING_DATASOURCE_URL	jdbc:h2:file:./data/...	Chemin de la base H2
JAVA_OPTS	-Xmx512m -Xms256m	Options JVM

13. Build & Déploiement

13.1 Build Maven

```
# Cloner le dépôt
git clone <url-du-depot>
cd soutenance-app

# Compiler et créer le JAR exécutable (tests ignorés)
mvn clean package -DskipTests

# JAR généré : target/soutenance-app-1.0.0-SNAPSHOT.jar
```

13.2 Exécution locale (développement)

```
# Via Maven (avec rechargement à chaud DevTools)
mvn spring-boot:run

# Via Java directement
java -jar target/soutenance-app-1.0.0-SNAPSHOT.jar

# Interface : http://localhost:8080
# Console H2 : http://localhost:8080/h2-console
```

14. Guide de développement

14.1 Setup de l'environnement

```
# Vérifier Java 17
java -version    # doit afficher "17"

# Vérifier Maven
mvn -version     # doit afficher "3.8+"

# Import dans IntelliJ IDEA
# File → Open → soutienance-app/pom.xml → Open as Project
# Activer Lombok : Settings → Build → Compiler → Annotation Processors → Enable
```

14.2 Annotations Lombok utilisées

Annotation	Effet généré
@Data	@Getter + @Setter + @ToString + @EqualsAndHashCode
@NoArgsConstructor	Constructeur sans arguments
@AllArgsConstructor	Constructeur avec tous les champs
@Builder	Pattern Builder fluide
@Slf4j	Champ log de type Logger

14.3 Ajouter une nouvelle entité

- Créer la classe dans entity/ avec les annotations JPA (@Entity, @Table, @Id, etc.)
- Créer l'interface repository dans repository/ étendant JpaRepository<Entité, Long>
- Créer l'interface service dans service/ et son implémentation dans service/impl/
- Injecter le service dans le contrôleur concerné
- Créer le template Thymeleaf dans resources/templates/
- Tester l'import via ExcellImportServiceImpl si besoin

14.4 Ajouter une règle de vérification

- Créer la classe dans service/verification/rules/ en implémentant VerificationRule
- Injecter les repositories nécessaires via le constructeur (@Autowired)
- Implémenter verifier() pour retourner List<Anomalie>
- Enregistrer la règle dans VerificationServiceImpl

14.5 Tests

```
# Lancer les tests unitaires
mvn test

# Tests d'intégration (démarré un contexte Spring complet)
mvn verify
```

14.6 Accès à la base de données (développement)

La console H2 est disponible à <http://localhost:8080/h2-console> avec :

Paramètre	Valeur
JDBC URL	<code>jdbc:h2:file:./data/soutenances</code>
Username	<code>sa</code>
Password	(vide)

15. Sécurité

⚠ ATTENTION

ATTENTION : Aucune authentification ni autorisation n'est implémentée dans la version actuelle. L'application est conçue pour un déploiement interne (intranet) uniquement.

15.1 État actuel

- Aucune dépendance `spring-boot-starter-security`
- Toutes les routes sont accessibles sans authentification
- La console H2 est exposée sur le réseau (désactiver en production)

15.2 Mesures minimales recommandées (production)

1. Désactiver la console H2

```
spring.h2.console.enabled=false
```

2. Ajouter Spring Security

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

# application-prod.properties
spring.security.user.name=admin
spring.security.user.password={bcrypt}$2a$10$...
spring.security.user.roles=ADMIN
```

3. Restreindre l'accès réseau

- Exposer l'application derrière un reverse proxy (Nginx / Apache)
- Restreindre l'accès au réseau interne de l'établissement

4. Sauvegardes quotidiennes de la base H2

```
# Sauvegarde quotidienne (cron)
cp /opt/soutenance/data/soutenances.mv.db \
  /backup/soutenances_$(date +%Y%m%d).mv.db
```

15.3 Validation des entrées

- Jakarta Bean Validation (@NotNull, @Size, etc.) disponible via spring-boot-starter-validation
- Import Excel protégé contre les fichiers malformés par des blocs try/catch avec journalisation
- Taille maximale des uploads limitée à 20 MB

16. Limitations connues & Évolutions futures

16.1 Limitations techniques actuelles

Limitation	Impact	Solution recommandée
Base H2 embarquée	Pas de concurrence multi-utilisateurs ; risque de corruption	Migrer vers PostgreSQL/MySQL
Pas de migration de schéma	ddl-auto=update est risqué en production	Intégrer Flyway ou Liquibase
Pas d'authentification	N'importe qui sur le réseau peut modifier les données	Intégrer Spring Security
Pas de Docker	Déploiement manuel	Créer Dockerfile + docker-compose.yml
Pas de CI/CD	Déploiements manuels, pas de qualité automatisée	Configurer GitHub Actions ou GitLab CI
Pas de tests métier	Régressions possibles lors des modifications	Ajouter des tests unitaires sur les services

16.2 Évolutions fonctionnelles suggérées

- Export par filière — permettre d'exporter le planning d'une seule filière
- Modification manuelle du planning — interface drag-and-drop
- Notifications par e-mail — envoi automatique des convocations
- Historique des années académiques — archivage des sessions précédentes
- Gestion des absences — signaler et remplacer un membre du jury absent
- Rapports personnalisés — exports filtrés par date, filière, professeur

16.3 Migration PostgreSQL recommandée

Pour une utilisation en production avec plusieurs utilisateurs simultanés, migrer vers PostgreSQL :

```
<!-- pom.xml -->
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>

# application-prod.properties
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/soutenances
spring.datasource.username=soutenance_user
spring.datasource.password=<mot-de-passe-securise>
spring.jpa.hibernate.ddl-auto=validate
spring.flyway.enabled=true
```

Annexe — Glossaire

Terme	Définition
PFE	Projet de Fin d'Études
Soutenance	Présentation orale du PFE devant un jury
Encadrant	Professeur responsable du suivi du projet de l'étudiant
Jury	Comité de 3 professeurs évaluant la soutenance
Président	Membre du jury présidant la séance
Examineur	Troisième membre du jury
Filière	Programme académique / spécialité (ex: Génie Informatique)
CNE	Code National de l'Étudiant — identifiant unique national
Créneau	Plage horaire d'une soutenance (ex: 09:00–10:00)
ENSAH	École Nationale des Sciences Appliquées d'Al-Hoceima
SSR	Server-Side Rendering — rendu HTML côté serveur
MVC	Model-View-Controller — pattern architectural
JPA	Jakarta Persistence API — standard ORM Java
ORM	Object-Relational Mapping — couche d'abstraction base de données
DTO	Data Transfer Object — objet de transfert de données
BOM	Bill of Materials — gestion centralisée des versions Maven